databricks

Data engineering with Lakeflow



Spark Do's & Don'ts workshop · companion talk

©2026 Databricks Inc. — All rights reserved

Agenda

The Databricks data engineering toolkit, end to end

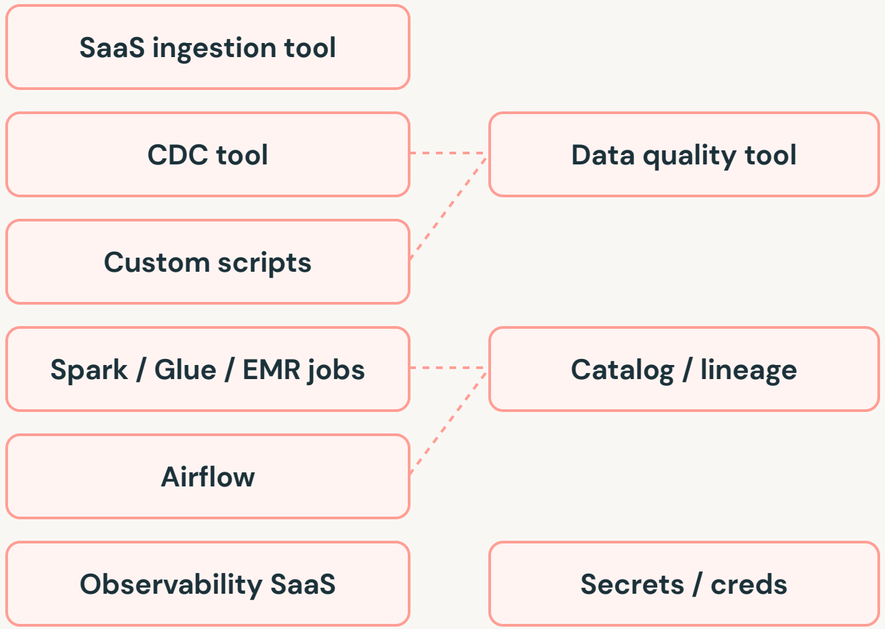
- Why one platform: what Lakeflow replaces
- **Ingest**: Lakeflow Connect, Auto Loader, Zerobus Ingest
- **Transform**: Spark Declarative Pipelines, the Pipelines Editor, real-time mode
- **Orchestrate**: Lakeflow Jobs
- **Build without code, build with agents**: Lakeflow Designer, Genie Code, Genie ZeroOps
- **Operate**: governance, quality, cost, CI/CD; when to use what

©2026 Databricks Inc. — All rights reserved

The job hasn't changed. The stack has.

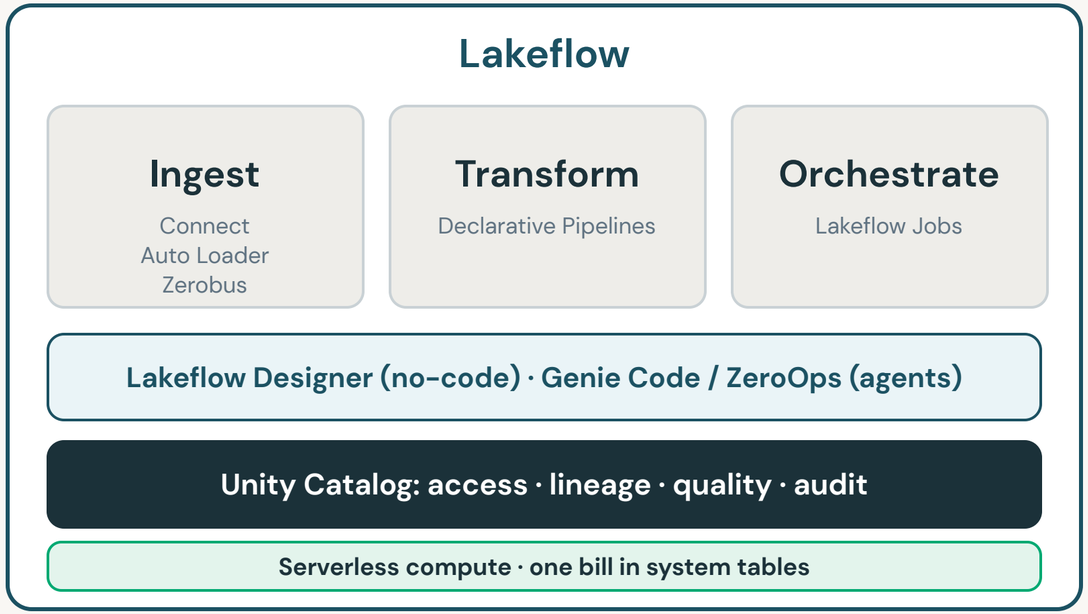
Get data from sources to consumers, reliably and on time

Stitched together



separate identity, lineage, monitoring and billing per tool

One platform

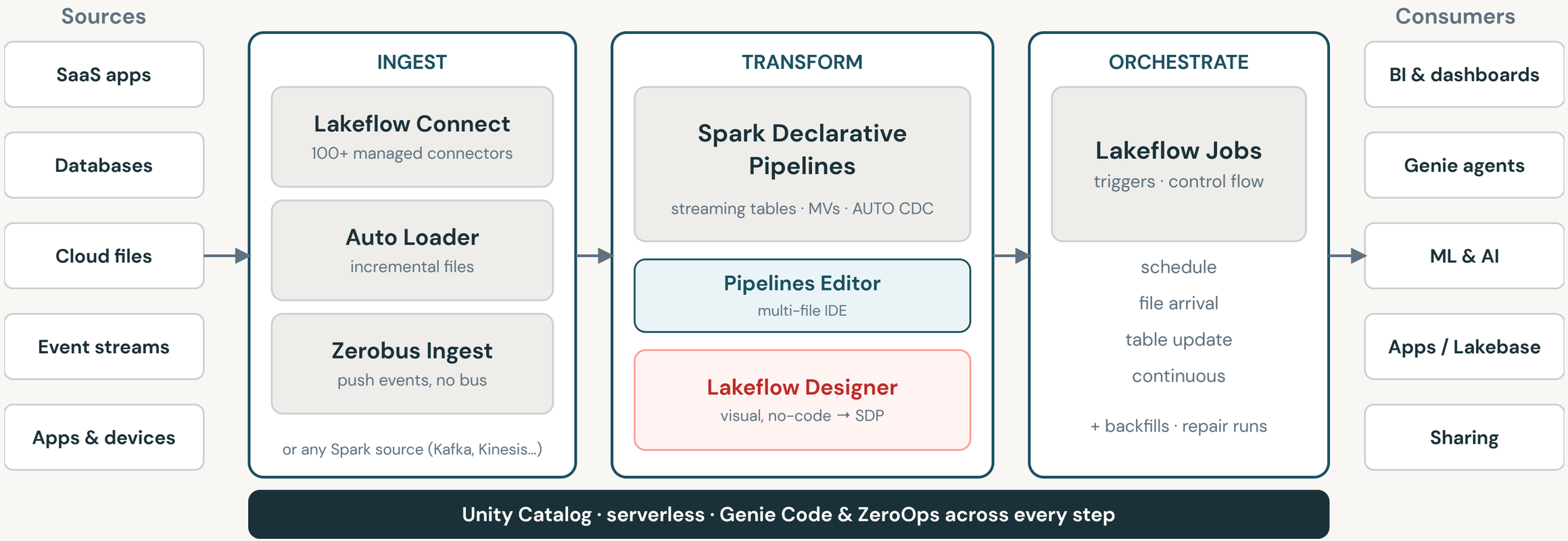


Every extra tool is another identity, another lineage gap, another bill and another on-call rotation.

©2026 Databricks Inc. — All rights reserved

Lakeflow at a glance

Ingest · transform · orchestrate, on Unity Catalog



©2026 Databricks Inc. — All rights reserved

PART 1

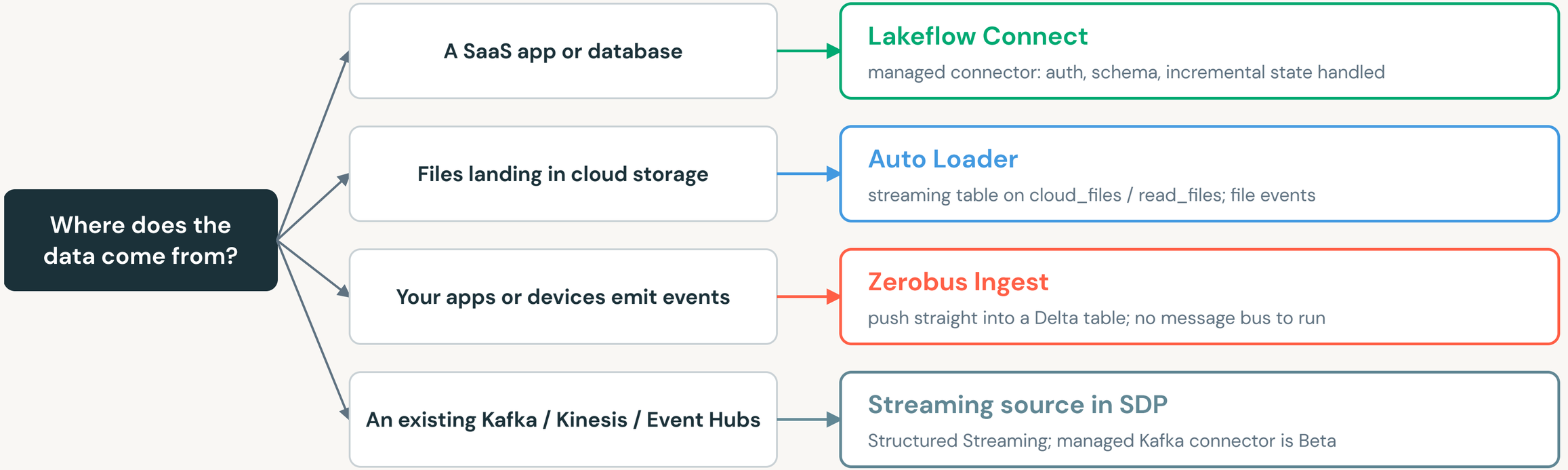
Ingest

1. Lakeflow Connect
2. Auto Loader
3. Zerobus Ingest
4. Picking the right one

©2026 Databricks Inc. — All rights reserved

Three ways in, one question

Pick the ingestion tool by where the data comes from



All four land in Unity Catalog Delta tables, so everything downstream looks the same.

©2026 Databricks Inc. — All rights reserved

Lakeflow Connect

Managed connectors for SaaS apps, databases and file sources

- **100+ managed connectors**: Salesforce, Workday, ServiceNow, SharePoint, SQL Server, Oracle, NetSuite, Google Analytics, Confluence, HubSpot…
- Incremental by default: each run lands only what changed, into UC streaming tables
- Databases: log-based **CDC**, or **query-based** capture on a cursor column (no gateway, no staging) GA
- **Community connectors**: open source, built on the Spark Python Data Source API Beta
- Runs on serverless; free tier of **100 DBUs per day** for managed connectors

Managed

UI, API or bundle; Databricks maintains the connector, schema handling and state.

Standard

`read_files`, Kafka, Kinesis, JDBC from SQL or Spark: full control, your code.

Federation

Query in place with Lakehouse Federation when you don't need a copy.

©2026 Databricks Inc. — All rights reserved

Auto Loader

Incremental, exactly-once file ingestion from cloud storage

```
(spark.readStream.format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", ckpt)
   .option("cloudFiles.useManagedFileEvents", "true")
   .load("/Volumes/.../raw_files/lab/orders_json")
   .writeStream.option("checkpointLocation", ckpt)
   .trigger(availableNow=True)
   .toTable("bronze_orders"))

-- or in SQL / a pipeline
SELECT * FROM STREAM read_files('/Volumes/...', format => 'json')
```

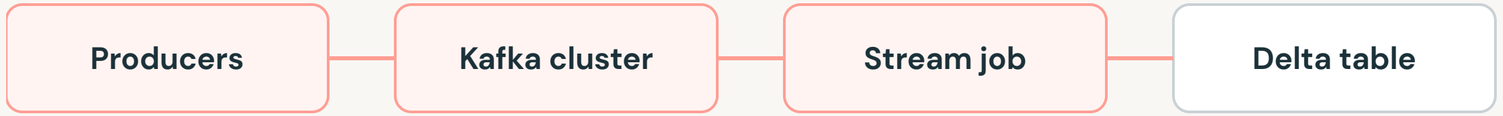
- Remembers which files it processed: rerun it and only new files are read
- Schema inference and **evolution**; bad fields go to `_rescued_data`, nothing silently lost
- **Managed file events** GA : no directory listing at scale
- `availableNow` = batch semantics with streaming bookkeeping

©2026 Databricks Inc. — All rights reserved

Zerobus Ingest

Push events straight into Delta tables, without running a message bus

Classic event path



a bus + a job to size, patch, monitor and secure

Zerobus Ingest



seconds to the table, nothing in between

- Serverless, part of Lakeflow Connect **GA Feb 2026**
- Advertised: under 5 s to the table, up to 100 MB/s per connection, 10+ GB/s per table
- Writes into tables or **streaming tables** **GA**
- gRPC and REST APIs, SDKs (Python, Java, Rust, Go, TypeScript) **GA**
- OpenTelemetry, MQTT and Kafka-compatible producer APIs: check status per interface
- Keep Kafka when many consumers need the same stream

©2026 Databricks Inc. — All rights reserved

PART 2

Transform

1. Spark Declarative Pipelines
2. Tables, flows and expectations
3. AUTO CDC
4. Incremental and real-time
5. The Pipelines Editor

©2026 Databricks Inc. — All rights reserved

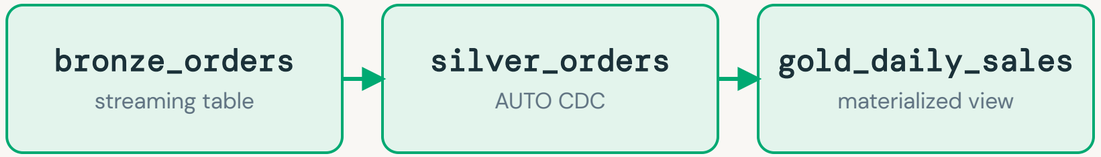
Spark Declarative Pipelines

Declare the tables you want; the engine works out how to keep them up to date

Imperative: you write the how

- read new files since checkpoint X
- parse + cast; route bad rows
- MERGE into silver by key, latest wins
- recompute gold for touched days
- order the jobs; retry; backfill

Declarative: you write the what



The engine works out

✓ dependency order and parallelism
✓ what is new since last run (incremental)
✓ checkpoints, retries, schema evolution
✓ compute sizing (serverless)

Formerly Delta Live Tables. Same idea, now part of Apache Spark itself.

©2026 Databricks Inc. — All rights reserved

Three building blocks

Choosing between a streaming table, a materialized view and a view

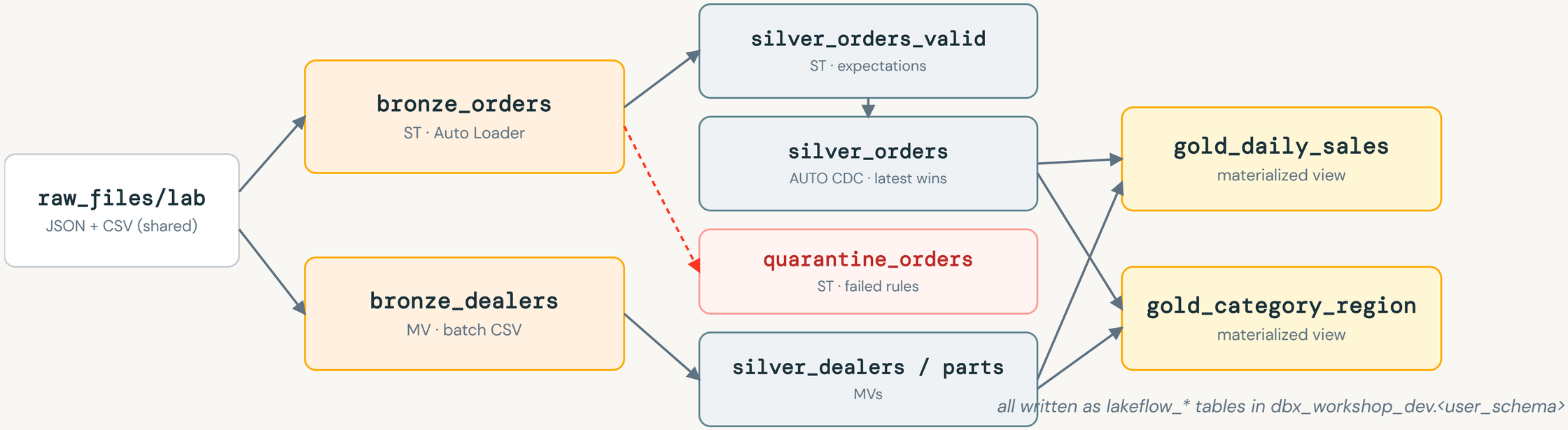
	WHAT IT IS	USE IT FOR	WORKSHOP EXAMPLE
Streaming table	Append-only, processes each input row once (incremental)	Ingestion, append-only facts, CDC targets	`lakeflow_bronze_orders_fact`
Materialized view	The result of a query, kept up to date; refreshed incrementally when possible	Joins, aggregates, gold tables	`lakeflow_gold_daily_dealer_sales_fact`
View	Not stored; computed when read	Intermediate logic you don't need to keep	an intermediate cleaning step
Flow	A query that writes into a target: append, AUTO CDC, update	Many sources into one table; backfills	`lakeflow_silver_orders_dedupe`

Streaming tables and materialized views also work outside a pipeline: create them in Databricks SQL or a notebook.

©2026 Databricks Inc. — All rights reserved

The bronze-to-gold lab, declaratively

The same tables you built by hand, as one pipeline



- **Bronze**: Auto Loader streaming table for orders (Python file); dealers as a batch materialized view
- **Silver**: expectations drop bad rows, a quarantine table keeps them with a reason, **AUTO CDC** keeps the latest re-sent order
- **Gold**: materialized views · all in `notebooks/jobs_notebooks/pipeline`

©2026 Databricks Inc. — All rights reserved

What the code looks like

Silver and gold from the workshop pipeline, in SQL

```
CREATE OR REFRESH STREAMING TABLE lakeflow_silver_orders_valid (
  CONSTRAINT known_dealer      EXPECT (dealer_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT positive_quantity EXPECT (quantity > 0)          ON VIOLATION DROP ROW
) AS SELECT try_cast(order_id AS BIGINT) AS order_id, ...
FROM STREAM(lakeflow_bronze_orders_fact);

CREATE OR REFRESH STREAMING TABLE lakeflow_silver_orders_fact;
CREATE FLOW lakeflow_silver_orders_dedupe AS AUTO CDC INTO lakeflow_silver_orders_fact
FROM STREAM(lakeflow_silver_orders_valid)
KEYS (order_id) SEQUENCE BY sent_at STORED AS SCD TYPE 1;

CREATE OR REFRESH MATERIALIZED VIEW lakeflow_gold_daily_dealer_sales_fact
AS SELECT to_date(o.order_ts) AS order_date, d.region, ..., sum(o.net_revenue) AS net_revenue
FROM lakeflow_silver_orders_fact o JOIN lakeflow_silver_dealers_dim d USING (dealer_id) GROUP BY ALL;
```

No checkpoint paths, no QUALIFY de-duplication, no ordering of steps: the engine derives all three.

©2026 Databricks Inc. — All rights reserved

Data quality with expectations

Declare what valid data looks like, and what to do when it isn't

ACTION	BAD ROWS	USE WHEN
`EXPECT`	kept; counted in metrics	you want visibility first
`… DROP ROW`	dropped; counted	bad rows must not flow on
`… FAIL UPDATE`	the update stops	bad data means a broken source

Quarantine pattern: a second flow selects the rows that fail the rule into their own table, as the lab's `lab_quarantine_orders_fact` does.

- Results land in the pipeline **event log**: pass/fail counts per rule, per update
- Delta `CHECK` constraints still guard writes from anywhere else
- Expectations are a Databricks extension (not in open-source SDP)
- Across tables: **data quality monitoring** watches freshness and completeness (part 5)

©2026 Databricks Inc. — All rights reserved

AUTO CDC

Apply a change feed correctly, with SCD type 1 or type 2, in a few lines

Change feed (arrives out of order)

`D042 · Gold · 11-15 08:00`

`D042 · Platinum · 11-20 09:30`

`D042 · Gold · 11-18 10:00`



SCD type 1: latest wins

`D042 · Platinum`

SEQUENCE BY handles late and out-of-order rows

SCD type 2: full history

`D042 · Gold 11-15 → 11-18`

`D042 · Gold 11-18 → 11-20`

`D042 · Platinum 11-20 → (current)`

__START_AT / __END_AT columns maintained for you

```python
from pyspark import pipelines as dp

dp.create_streaming_table("lakeflow_silver_orders_fact")
dp.create_auto_cdc_flow(
    target="lakeflow_silver_orders_fact",
    source="lakeflow_silver_orders_valid",
    keys=["order_id"], sequence_by="sent_at",
    stored_as_scd_type=1)   # 2 keeps full history
```

- Handles **out-of-order** and late changes with `sequence_by`
- Deletes via `apply_as_deletes`; `AUTO CDC FROM SNAPSHOT` for full snapshots
- Formerly `APPLY CHANGES`; SQL form on the previous slide

©2026 Databricks Inc. — All rights reserved

Incremental by default, real-time when you need it

How much work each update does, and how fast

Incremental MVs

- Serverless pipelines refresh materialized views incrementally when the query allows
- Falls back to a full recompute only when it must

Incremental Replace Where

- Batch semantics (replace a slice) without recomputing everything GA
- Good fit for daily re-processing

Real-Time Mode

- Structured Streaming GA Mar 2026
- Declarative pipelines GA Oct 2026
- Millisecond-scale latency, classic and serverless

– Same code, different trigger: **triggered** (batch-like, cheapest) → **continuous** (seconds) → **real-time mode** (ms)

– Declarative APIs (append, AUTO CDC, Replace Where, MVs) are also available from notebooks and the SQL editor

©2026 Databricks Inc. — All rights reserved

Lakeflow Pipelines Editor

A multi-file IDE built for pipelines

- Pipeline as a **folder of Python and SQL files**, in a Git folder
  GA May 2026
- Live **pipeline graph** and data previews next to the code
- **Run one file or one table**: dry runs and selective refresh while you iterate
- Issues panel and execution insights per table
- **Genie Code** in the editor: create, change and debug the pipeline

Typical loop

- Write a table in a new file
- Run just that table; check preview and expectations
- Commit from the Git folder
- Deploy with a bundle to test, then prod

©2026 Databricks Inc. — All rights reserved

PART 3

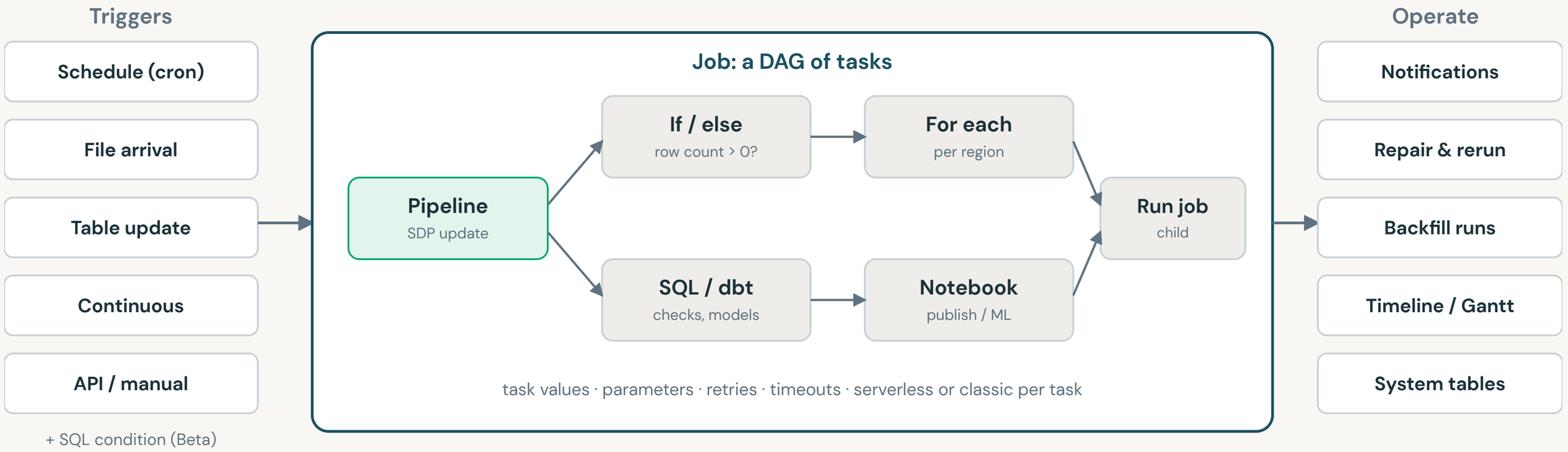
Orchestrate

1. Lakeflow Jobs
2. Triggers
3. Control flow and operations

©2026 Databricks Inc. — All rights reserved

Lakeflow Jobs

One orchestrator for pipelines, notebooks, SQL, dbt, dashboards and ML



©2026 Databricks Inc. — All rights reserved

Triggers: run when the data is ready

Event-driven beats a cron guess

TRIGGER	STARTS A RUN WHEN…	NOTES
Schedule	a cron time is reached	the default; pair with a condition below
File arrival	new files land in a UC volume or external location	no polling code
Table update	monitored tables change (any / all of them)	updated tables passed to tasks GA
Continuous	the previous run ends	always-on streaming jobs
SQL condition	a query on a SQL warehouse returns rows or a new value	Beta
Multiple triggers	up to five per job, e.g. table update + nightly safety run	Beta

©2026 Databricks Inc. — All rights reserved

Control flow and day-2 operations

What makes a job production-ready

Control flow

- **If / else** on task values or results
- **For each** over a list, with concurrency
- **Run job** for reusable child jobs
- Job and task parameters, task values

Recovery

- Retries and timeouts per task
- **Repair run**: rerun failed tasks only
- **Backfill**: one job over a date range, up to 100 runs
- Partial runs while debugging

Visibility

- Notifications: email, Slack, webhooks
- Duration alerts for late runs
- Timeline view of the critical path
- `system.lakeflow` tables for runs and cost

– Outside Databricks: an open operator framework for external systems (REST, Snowflake, Slack, PagerDuty) **Announced**

– Coming from Airflow or ADF? The concepts map one-to-one: DAG → job, operator → task, sensor → trigger

©2026 Databricks Inc. — All rights reserved

PART 4

Without code, and with agents

1. Lakeflow Designer
2. Genie Code
3. Genie ZeroOps

©2026 Databricks Inc. — All rights reserved

Lakeflow Designer

Visual, AI-assisted data prep that produces real pipelines

- Drag-and-drop canvas or plain-English prompts **GA Jun 2026**
- Runs on Spark Declarative Pipelines; outputs are UC tables or **materialized views**
- Generated code visible per operator; files live in Git folders
- Parameterized sources and outputs: one flow, test and prod schedules
- October 2026: Prepare, Unique and **Guardrails** (quality) operators; multi-join; file outputs

Who it's for

Analysts and domain experts who'd otherwise use a desktop prep tool, and the engineers who review and productionize their flows.

Where it stops

Complex ETL with custom logic still belongs in code; Designer has Python and SQL operators for the gaps.

©2026 Databricks Inc. — All rights reserved

Agents across the lifecycle

Genie Code builds; Genie ZeroOps keeps it running

Genie Code GA

- Creates ingestion connectors, Python and SQL pipelines, and jobs with triggers
- Grounded in Unity Catalog: knows your schemas, lineage and permissions
- Writes **declarative** pipelines, so the engine validates dependencies before anything runs
- You used it in the workshop: the scorecard challenge

Genie ZeroOps Private Preview

- Background agent for jobs, pipelines and tables
- Detects failures and quality anomalies; root cause through lineage
- Proposes a fix, tests it in a sandbox, opens it for review
- Nothing reaches production without human approval

©2026 Databricks Inc. — All rights reserved

PART 5

Operate

1. Governance and quality
2. Cost and compute
3. CI/CD and migration
4. When to use what

©2026 Databricks Inc. — All rights reserved

Governed and observable by default

Unity Catalog sees every table, pipeline and job

Lineage

- Table and column lineage from source to dashboard, captured automatically
- Jobs and pipelines linked to the tables they write

Data quality monitoring

- Anomaly detection on **freshness** and **completeness**, per schema GA Oct 2026
- Learns each table's normal pattern; no rules to write

Cost and runs

- `system.billing`, `system.lakeflow` tables
- Serverless: **performance-optimized** or **standard** mode per job
- Tag jobs and pipelines to attribute cost

– Same permissions model for connectors, pipelines, tables and agents: grant once in Unity Catalog

©2026 Databricks Inc. — All rights reserved

CI/CD and migration

Treat pipelines and jobs as code

```
# databricks.yml: one bundle, many targets
resources:
  pipelines:
    lab_pipeline:
      catalog: dbx_workshop_dev
      schema: ${var.user_schema}
      libraries: [{glob: {include: pipeline/**}}]
  jobs:
    lab_job:
      tasks: [{task_key: refresh,
               pipeline_task: {pipeline_id: ${resources.pipelines.lab_pipeline.id}}}]

$ databricks bundle deploy -t prod
```

- **Declarative Automation Bundles** (formerly Databricks Asset Bundles, renamed Mar 2026): YAML or Python
- Git folders for development; CLI, REST, SDKs and Terraform for automation
- **Lakebridge**: free tooling to assess and convert legacy ETL and SQL
- This workshop ships as a bundle: `databricks.yml` + `resources/jobs.yml`

©2026 Databricks Inc. — All rights reserved

When to use what

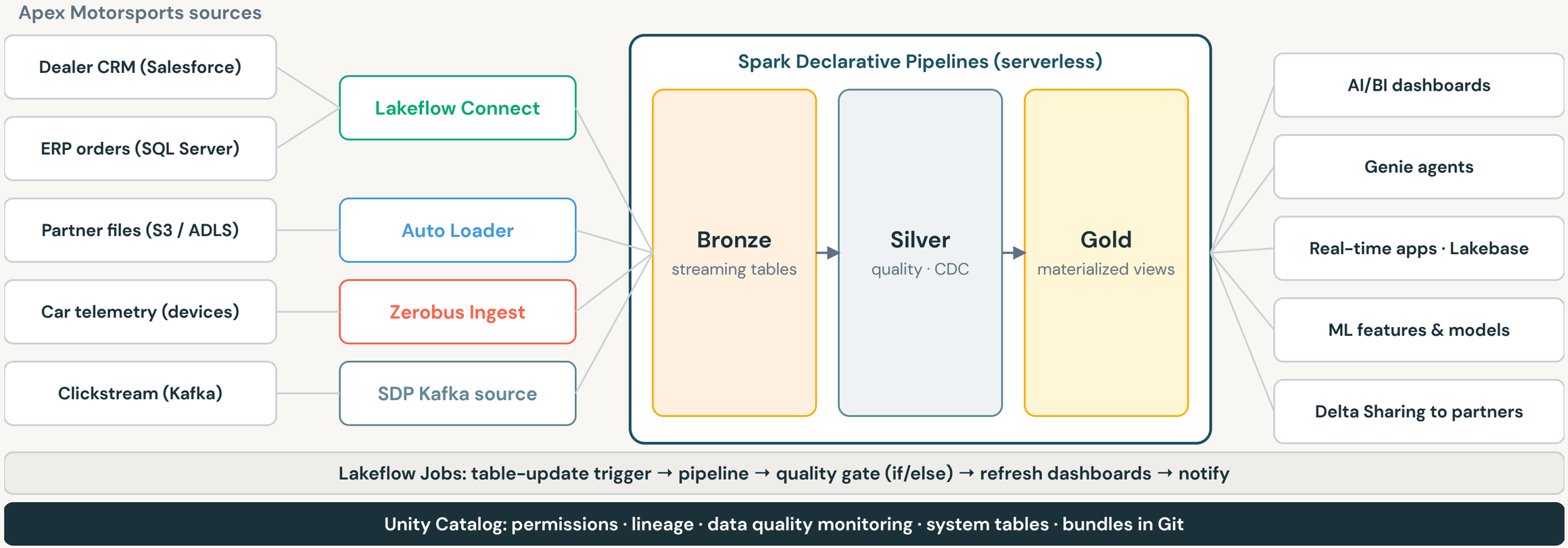
A cheat sheet for the common choices

YOU NEED TO…	USE	INSTEAD OF
Copy a SaaS app or database into the lakehouse	Lakeflow Connect	custom API / CDC scripts, a separate ingestion tool
Pick up files as they land	Auto Loader (in a pipeline)	full reloads, listing folders yourself
Get app or device events into tables	Zerobus Ingest	a Kafka cluster used only as a pipe
Build medallion tables, batch or streaming	Spark Declarative Pipelines	notebooks with hand-rolled MERGE and checkpoints
Apply CDC or keep SCD2 history	AUTO CDC	MERGE with ROW_NUMBER de-duplication
Run things in order, on events	Lakeflow Jobs + triggers	an external orchestrator, cron guesses
Let analysts prep data themselves	Lakeflow Designer	desktop prep tools and file hand-offs
One-off exploration or a custom step	a notebook (as a job task)	forcing it into a pipeline

©2026 Databricks Inc. — All rights reserved

Reference architecture

Apex Motorsports Parts, end to end on Lakeflow



©2026 Databricks Inc. — All rights reserved

Where this shows up in the workshop

Everything on these slides you can try today

01 · bronze-to-gold lab

- Auto Loader into bronze
- QUALIFY de-duplication, CHECK constraints, quarantine
- Gold tables and a dashboard, by hand

jobs_notebooks

- The same lab as a **declarative pipeline**: Auto Loader, expectations, AUTO CDC, quarantine, MVs
- One notebook creates the pipeline and a **Lakeflow job** in your schema

02 · do's and don'ts

- 6c: full reload vs Auto Loader
- 9: liquid clustering and file layout
- 11: Genie Code optimizes a slow job

genie/

- Genie agents and metric views on the gold tables the pipeline produces

©2026 Databricks Inc. — All rights reserved

Status at a glance

What's GA, what's preview, as of November 2026

CAPABILITY	STATUS	SINCE
Lakeflow Connect managed connectors · query-based capture	GA	connector-by-connector; many SaaS ones still Beta
Community connectors · managed Kafka connector	Beta	2026
Zerobus Ingest (gRPC, REST, SDKs)	GA	Feb 2026
Auto Loader with managed file events	GA	Dec 2025
Pipelines Editor	GA	May 2026
Real-time mode in declarative pipelines	GA	Oct 2026 (Structured Streaming: Mar 2026)
Lakeflow Designer	GA	Jun 2026
Jobs: SQL-condition triggers · multiple triggers	Beta	2026
Data quality monitoring (freshness, completeness)	GA	Oct 2026
Genie ZeroOps	Private Preview	announced Jun 2026

©2026 Databricks Inc. — All rights reserved



©2026 Databricks Inc. — All rights reserved